


```
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: web-demo
5    namespace: dev
6  spec:
7    selector:
8      matchLabels:
9        app: web-demo
10   replicas: 1
11   template:
12     metadata:
13       labels:
14         app: web-demo
15     spec:
16       containers:
17         - name: web-demo
18           image: hub.mooc.com/kubernetes/web:v1
19           ports:
20             - containerPort: 8080
21           livenessProbe:
22             tcpSocket:
23               port: 8080
24             initialDelaySeconds: 20 # 容器启动多久后执行检查
25             periodSeconds: 10
26             failureThreshold: 2
27             successThreshold: 1
28             timeoutSeconds: 5
29           readinessProbe:
30             tcpSocket:
31               port: 8080
32             initialDelaySeconds: 20 # 容器启动多久后执行检查
33             periodSeconds: 10
34             failureThreshold: 2
35             successThreshold: 1
36             timeoutSeconds: 5
37
```

几个参数的含义

initialDelaySeconds: 20 # 容器启动多久后执行检查

periodSeconds: 10 # 每10秒检查一次

failureThreshold: 2 # 两次检查出错就认为检查不通过

successThreshold: 1 # 一次检查成功就认为检查成功

timeoutSeconds: 5 # 执行tcp检查时的超时时间，防止执行命令一直卡住

kubelet 使用存活探测器（livenessProbe）来知道什么时候要重启容器。例如，存活探测器可以捕捉到死锁（应用程序在运行，但是无法继续执行后面的步骤）。这样的情况下重启容器有助于让应用程序在有问题的情况下更可用。

kubelet 使用就绪探测器（readinessProbe）可以知道容器什么时候准备好了并可以开始接受请求流量，当一个 Pod 内的所有容器都准备好了，才能把这个 Pod 看作就绪了。这种信号的一个用途就是控制哪个 Pod 作为 Service 的后端。在 Pod 还没有准备好的时候，会从 Service 的负载均衡器中被剔除的。

kubelet 使用启动探测器（startupProbe）可以知道应用程序容器什么时候启动了。如果配置了这类探测器，就可以控制容器在启动成功后再进行存活性和就绪检查，确保这些存活、就绪探测器不会影响应用程序的启动。这可以用于对慢启动容器进行存活性检测，避免它们在启动运行之前就被杀掉。